SQL Fundamentals

Level 1 - Concepts, Examples and Explanations

All examples use MySQL syntax

What you will learn

- Introduction to SQL, DBMS vs RDBMS, SQL vs MySQL
- The five SQL command categories: DDL, DML, DQL, DCL, TCL
- Creating databases and tables, and choosing data types
- CREATE, ALTER, DROP, TRUNCATE, RENAME
- INSERT, UPDATE, DELETE, SELECT
- Practice solutions (Q1 to Q15) and a DELETE vs DROP vs TRUNCATE comparison

Sample table used throughout: `Students(student_id, name, age, gender, course, marks)`

1. Introduction to SQL

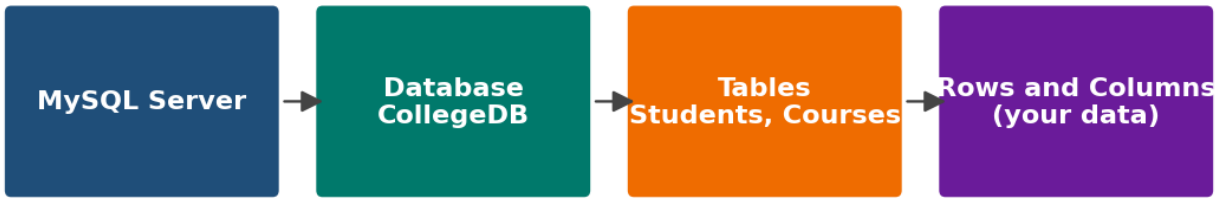
SQL (Structured Query Language) is the standard language used to talk to a relational database. With SQL you can store data, read it, change it, delete it, and control who is allowed to access it.

Example

```
SELECT name FROM Students;
```

Explanation

This asks the database: *give me the **name** column from the **Students** table*. SQL reads almost like English: **select** this **from** that.



A server holds databases, a database holds tables, a table holds rows of data

Figure 1: How data is organised: server, database, tables, rows and columns

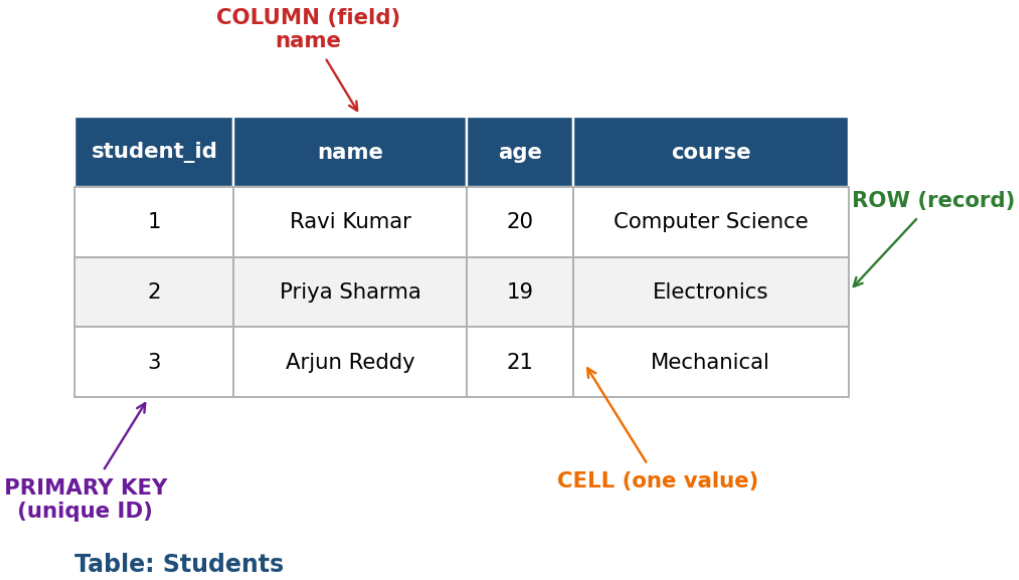


Figure 2: Parts of a table

2. DBMS and RDBMS

DBMS (Database Management System) is software that stores and manages data. The data may be kept in plain files with no links between them.

RDBMS (Relational DBMS) stores data in **tables** (rows and columns) and connects tables using **keys**. MySQL, PostgreSQL, Oracle and SQL Server are RDBMS.

	DBMS	RDBMS
Data storage	Files or other formats	Tables with rows and columns
Relationships	Usually none	Tables linked by primary and foreign keys
Normalization	Not supported	Supported
Examples	File systems, XML stores	MySQL, PostgreSQL, Oracle, SQL Server

Example (the relational idea)

```
Students(student_id, name, course_id)
Courses(course_id, course_name)
```

Explanation

`course_id` appears in both tables. Instead of typing "Computer Science" in every student's row, each student stores only a number (for example 101). The course name lives in one place, so it is stored once and is easy to update. This link between tables is what makes it *relational*.

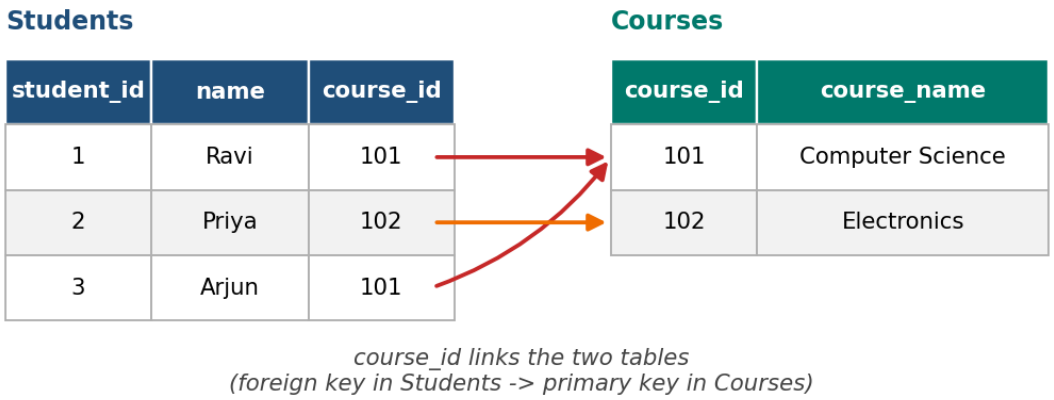


Figure 3: Two tables linked by course_id

3. SQL vs MySQL

SQL	MySQL
A language	A software product (database server)
The standard, like English grammar	One product that follows it, like a person who speaks English
Used by many databases	Developed as an RDBMS that runs SQL

Example

```
SELECT VERSION();
```

Explanation

`SELECT` is standard SQL. `VERSION()` is a function provided by MySQL that returns the server's version. Other databases (like Oracle) understand most SQL but have their own extra features.

4. SQL Command Categories

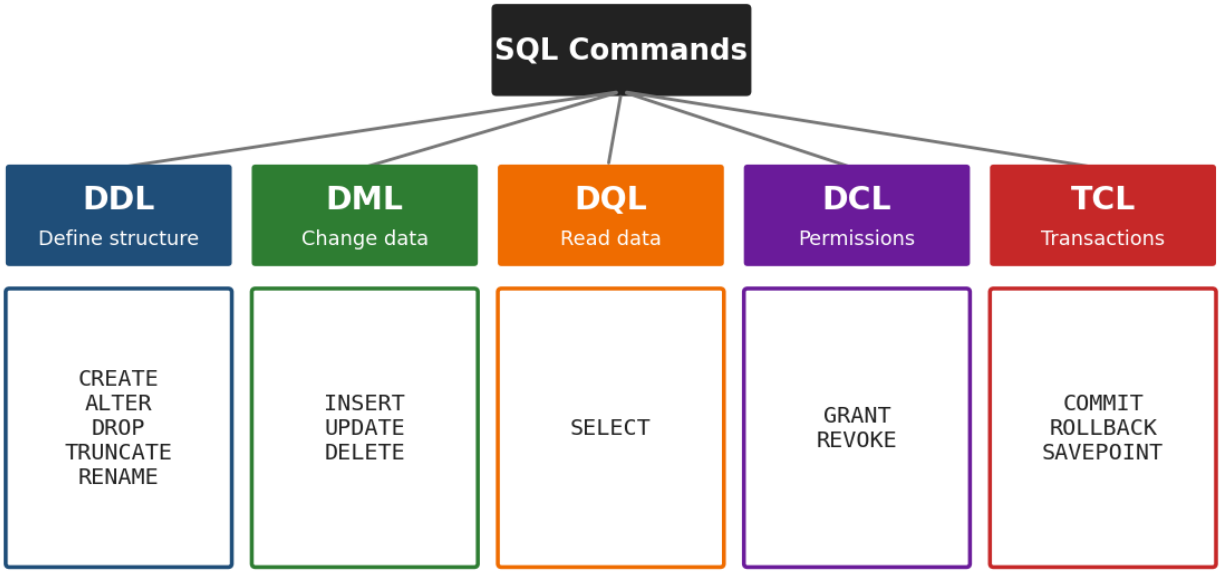


Figure 4: The five categories of SQL commands

DDL - Data Definition Language

Defines the **structure** of the database. Commands: CREATE, ALTER, DROP, TRUNCATE, RENAME.

```
CREATE TABLE Demo (id INT, title VARCHAR(30));
```

Explanation: builds a new table named Demo. DDL changes the *shape* of the database, not just the data inside.

DML - Data Manipulation Language

Changes the **data** inside tables. Commands: INSERT, UPDATE, DELETE.

```
INSERT INTO Demo VALUES (1, 'Hello');
```

Explanation: adds one row (1, 'Hello') into the Demo table.

DQL - Data Query Language

Reads data without changing it. The only command is SELECT.

```
SELECT * FROM Demo;
```

Explanation: shows every row and column of Demo. Nothing is modified.

DCL - Data Control Language

Controls **permissions**. Commands: GRANT, REVOKE.

```
GRANT SELECT ON CollegeDB.Students TO 'teacher'@'localhost';
REVOKE SELECT ON CollegeDB.Students FROM 'teacher'@'localhost';
```

Explanation: GRANT lets the user *teacher* read the Students table. REVOKE takes that permission back.

TCL - Transaction Control Language

Manages **transactions**, which are groups of changes treated as one unit. Commands: COMMIT, ROLLBACK, SAVEPOINT.

```
START TRANSACTION;
UPDATE Students SET marks = 50 WHERE student_id = 1;
ROLLBACK;
```

Explanation: START TRANSACTION begins a group of changes. ROLLBACK undoes everything since then, so the marks return to their old value. COMMIT would have made the change permanent. SAVEPOINT marks a point you can roll back to.

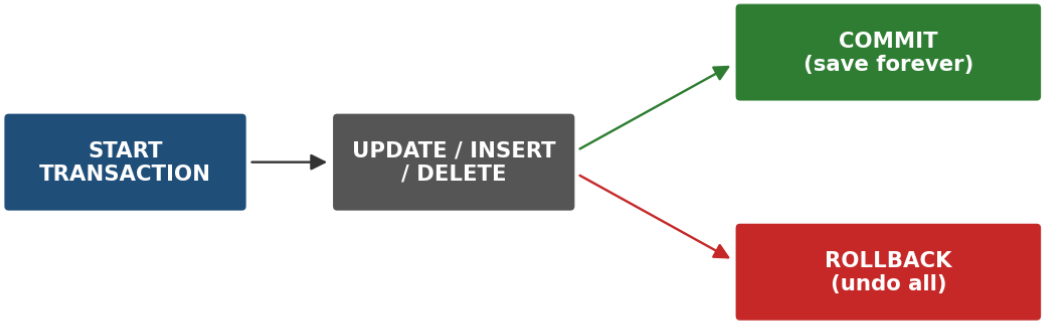


Figure 5: Life of a transaction

5. Database Creation

A **database** is a container that holds tables.

Example

```
CREATE DATABASE CollegeDB;
USE CollegeDB;
SHOW DATABASES;
```

Explanation

- `CREATE DATABASE` makes the empty container.
- `USE` selects it, so later commands apply to it.
- `SHOW DATABASES` lists all databases so you can confirm it exists.

6. Table Creation

A **table** stores data in rows (records) and columns (fields).

Example

```
CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    name       VARCHAR(50) NOT NULL,
    age        INT,
    gender     VARCHAR(10),
    course     VARCHAR(50),
    marks      DECIMAL(5,2)
);
```

Explanation

- `student_id INT PRIMARY KEY`: a whole-number column; every value must be unique and not empty. It identifies each student.
- `name VARCHAR(50) NOT NULL`: text up to 50 characters that cannot be left blank.
- `age INT`: a whole number; optional.
- `marks DECIMAL(5,2)`: up to 5 digits in total, 2 after the decimal point (for example 85.50).

7. Data Types

A **data type** tells the database what kind of value a column can hold.

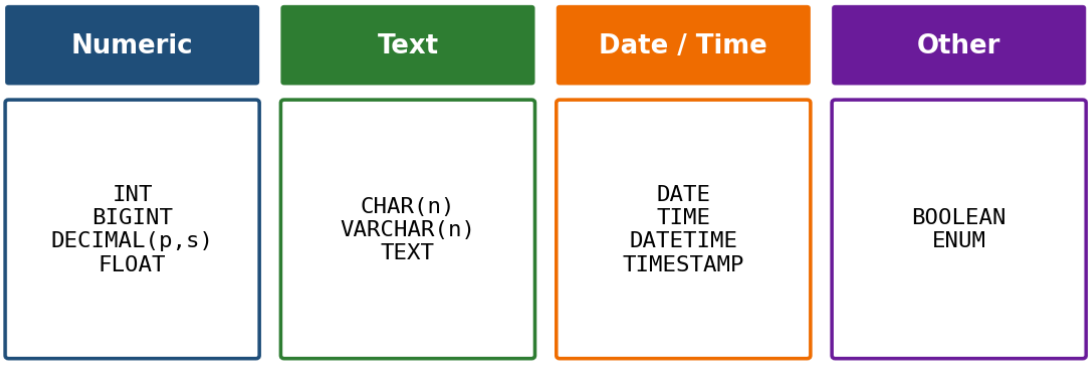


Figure 6: Common MySQL data types

Example

```
CREATE TABLE DataTypeDemo (
    id         INT,
    name       VARCHAR(30),
    grade      CHAR(1),
    fees       DECIMAL(8,2),
    birth_date DATE,
    is_active  BOOLEAN
);
```

Explanation

Column	Type	Meaning
id	INT	Whole numbers (1, 2, 100)
name	VARCHAR(30)	Text of variable length, max 30 characters
grade	CHAR(1)	Fixed-length text, exactly 1 character (like 'A')
fees	DECIMAL(8,2)	Up to 8 digits, 2 after the point (e.g. 12500.50)
birth_date	DATE	A date in YYYY-MM-DD format
is_active	BOOLEAN	True or false (stored as 1 or 0 in MySQL)

8. CREATE

Builds new objects such as databases and tables.

Example

```
CREATE TABLE Courses (
    course_id   INT PRIMARY KEY AUTO_INCREMENT,
    course_name VARCHAR(50) NOT NULL,
    duration    INT,
    fees        DECIMAL(10,2)
);
```

Explanation

This makes a **Courses** table. `AUTO_INCREMENT` means you do not supply `course_id`; MySQL assigns 1, 2, 3... automatically for each new row.

9. ALTER

Changes the **structure** of an existing table.

Examples

```
ALTER TABLE Students ADD email VARCHAR(100);
ALTER TABLE Students MODIFY age TINYINT;
ALTER TABLE Students RENAME COLUMN name TO student_name;
ALTER TABLE Students DROP COLUMN email;
```

Explanation

- **ADD** puts a new *email* column in the table. Existing rows get NULL there.
- **MODIFY** changes the datatype of *age* from INT to the smaller TINYINT.
- **RENAME COLUMN** changes the column's name but keeps its data (MySQL 8.0+; older versions use CHANGE).
- **DROP COLUMN** deletes the column and all its data.

Older MySQL rename:

```
ALTER TABLE Students CHANGE name student_name VARCHAR(50);
```

10. DROP

Permanently deletes a database or table, including its structure and data.

Example

```
DROP TABLE Courses;
DROP DATABASE CollegeDB;
DROP TABLE IF EXISTS Courses;
```

Explanation

After `DROP TABLE Courses` the table no longer exists, so you cannot SELECT from it. `DROP DATABASE` removes the whole database with every table inside. **This cannot be undone.** The `IF EXISTS` version gives no error when the table is missing.

11. TRUNCATE

Removes **all rows** from a table but keeps the table itself.

Example

```
TRUNCATE TABLE Students;
SELECT * FROM Students;
```

Explanation

After truncating, the second query returns an empty result, but the table and its columns are still there, ready for new inserts. Auto-increment counters restart from 1. There is no WHERE clause, so it is all or nothing.

12. RENAME

Changes the name of a table (or a column).

Example

```
RENAME TABLE Students TO College_Students;
-- or
ALTER TABLE College_Students RENAME TO Students;
```

Explanation

The first line changes the table name from *Students* to *College_Students*. All data stays intact. The second line shows another way to do the same thing and here renames it back.

13. INSERT

Adds new rows to a table.

Example

```
-- Single row
INSERT INTO Students (student_id, name, age, gender, course, marks)
VALUES (1, 'Ravi Kumar', 20, 'Male', 'Computer Science', 85.50);

-- Multiple rows
INSERT INTO Students (student_id, name, age, gender, course, marks) VALUES
(2, 'Priya Sharma', 19, 'Female', 'Electronics', 78.00),
(3, 'Arjun Reddy',  21, 'Male',   'Mechanical',  66.25);
```

Explanation

The first bracket lists the columns. VALUES gives the data in the same order. Text and dates go in single quotes; numbers do not. Inserting several rows in one statement is faster than writing separate statements.

14. UPDATE

Changes existing data in a table.

Example

```
UPDATE Students
SET marks = 90.00, course = 'Electronics'
WHERE student_id = 3;
```

Explanation

This finds the student whose `student_id` is 3 and sets both marks and course. Separate several columns with commas. **Always include WHERE.** Without it, every student's marks and course would be overwritten.

15. DELETE

Removes specific rows from a table.

Examples

```
DELETE FROM Students WHERE student_id = 9;    -- one row
DELETE FROM Students WHERE marks < 60;        -- rows matching a condition
DELETE FROM Students;                         -- all rows (no WHERE!)
```

Explanation

- Removes only the student with ID 9.
- Removes every student scoring below 60.
- With no WHERE, removes every row, but the table structure stays.
- Because DELETE is DML, it can be rolled back inside a transaction.

16. SELECT

Retrieves data from a table. It is the most-used SQL command.

Examples

```
SELECT * FROM Students;                        -- all columns, all rows
SELECT name, marks FROM Students;              -- selected columns
SELECT * FROM Students WHERE marks > 80;       -- filtered rows
SELECT DISTINCT course FROM Students;          -- unique values only
SELECT * FROM Students ORDER BY marks DESC;    -- sorted, highest first
SELECT name AS student_name FROM Students;     -- column alias
```

Explanation

- ***** means all columns.
- Listing column names returns only those columns.
- **WHERE** filters rows by a condition.
- **DISTINCT** removes duplicates, so each course appears once.
- **ORDER BY ... DESC** sorts highest to lowest (ASC, the default, sorts lowest to highest).
- **AS** gives the output column a friendlier temporary name. The table itself is not changed.

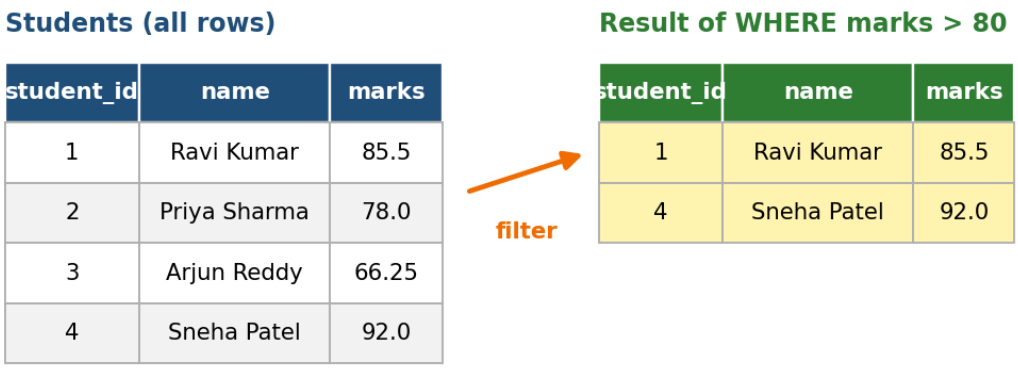


Figure 7: SELECT with WHERE keeps only rows that match the condition

Practice Questions with Solutions

Q1. Create a database called CollegeDB.

```
CREATE DATABASE CollegeDB;
USE CollegeDB;
```

Q2 and Q3. Create the Students table with student_id, name, age, gender, course, marks.

```
CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    name       VARCHAR(50) NOT NULL,
    age        INT,
    gender     VARCHAR(10),
    course     VARCHAR(50),
    marks      DECIMAL(5,2)
);
```

Q4. Insert 10 student records.

```
INSERT INTO Students (student_id, name, age, gender, course, marks) VALUES
(1,  'Ravi Kumar',   20, 'Male',   'Computer Science', 85.50),
(2,  'Priya Sharma', 19, 'Female', 'Electronics',      78.00),
(3,  'Arjun Reddy',  21, 'Male',   'Mechanical',       66.25),
(4,  'Sneha Patel',  20, 'Female', 'Computer Science', 92.00),
(5,  'Kiran Rao',    22, 'Male',   'Civil',            71.75),
(6,  'Ananya Das',   19, 'Female', 'Electronics',      88.40),
(7,  'Vikram Singh', 21, 'Male',   'Mechanical',       59.90),
(8,  'Meena Iyer',   20, 'Female', 'Computer Science', 81.00),
(9,  'Rahul Verma',  23, 'Male',   'Civil',            64.50),
(10, 'Divya Nair',   19, 'Female', 'Electronics',      95.20);
```

Q5. Display all students.

```
SELECT * FROM Students;
```

Q6. Display only student names.

```
SELECT name FROM Students;
```

Q7. Update a student's marks.

```
UPDATE Students SET marks = 90.00 WHERE student_id = 3;
```

Q8. Update a student's course.

```
UPDATE Students SET course = 'Computer Science' WHERE student_id = 5;
```

Q9. Delete one student.

```
DELETE FROM Students WHERE student_id = 9;
```

Q10. Add an email column.

```
ALTER TABLE Students ADD email VARCHAR(100);
```

Q11. Modify a column datatype.

```
ALTER TABLE Students MODIFY age TINYINT;
```

Q12. Rename a column.

```
-- MySQL 8.0+
ALTER TABLE Students RENAME COLUMN name TO student_name;

-- Older MySQL
ALTER TABLE Students CHANGE name student_name VARCHAR(50);
```

Q13. Create a Courses table.

```
CREATE TABLE Courses (
    course_id   INT PRIMARY KEY AUTO_INCREMENT,
    course_name VARCHAR(50) NOT NULL,
    duration    INT,
    fees        DECIMAL(10,2)
);
```

Q14. Truncate the Students table.

```
TRUNCATE TABLE Students;
```

Q15. Explain DELETE, DROP and TRUNCATE

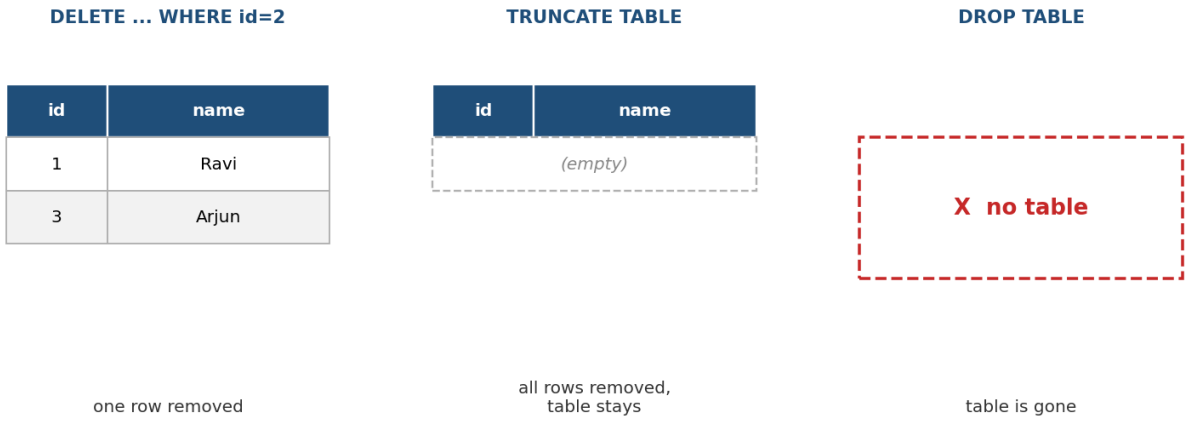


Figure 8: What each command does to a table

Feature	DELETE	TRUNCATE	DROP
Category	DML	DDL	DDL
What it removes	Selected or all rows	All rows	Entire table (structure and data)
WHERE clause	Yes	No	No
Table structure	Kept	Kept	Removed
Rollback	Yes, inside a transaction	Generally no (implicit commit in MySQL)	No
Speed	Slower (row by row)	Fast	Fast
Auto-increment counter	Not reset	Reset	Table is gone

```
DELETE FROM Students WHERE student_id = 2;  -- removes ONE row, table stays
TRUNCATE TABLE Students;                    -- removes ALL rows, table stays
DROP TABLE Students;                        -- removes the TABLE itself
```

Analogy: think of a notebook. DELETE erases specific lines, TRUNCATE erases every page but keeps the notebook, and DROP throws the notebook away.